

CodeFlow (Code Visualizer) Architectural Deep-Dive

This document provides a comprehensive, end-to-end breakdown of the architecture, data flows, execution mechanics, and design patterns of **CodeFlow** (Code Visualizer). It is structured to help developers, interviewers, recruiters, and founders understand exactly how the system works.

For every section, you will find:

1. **Simple Explanation:** Written in plain, non-technical language.
2. **Technical Explanation:** Deep-dive implementation details mapping directly to files in the repository.
3. **Real-World Analogy:** Connects the concept to everyday situations.
4. **Example Flow:** A step-by-step trace of how data flows through the system.
5. **Interview Questions & Answers:** Expected technical questions and standard answers.

1. PROJECT OVERVIEW

Simple Explanation

When beginners learn to write code, it feels like typing magical spells in the dark. They run the program, and either it works, or they get a confusing error message. They cannot "see" inside the computer's brain. If they write a loop to reverse an array, they have to imagine the variables swapping values in their head.

CodeFlow is like giving developers X-ray glasses. Instead of forcing you to guess how your code is running, CodeFlow translates code into an interactive, step-by-step visual animation. When your code swaps two numbers, you actually see the boxes slide past each other. When

your code makes a recursive call, you see a tree branch grow. It solves the "black box" problem of coding, making learning Data Structures and Algorithms (DSA) intuitive.

Technical Explanation

Traditional online compilers (like LeetCode, HackerRank, or standard IDEs) compile source code into machine instructions or run them inside an interpreter process, capture the final console output (``stdout`/`stderr``), and display it. They do not capture execution state transitions.

CodeFlow captures state transitions. It parses user code into an **Abstract Syntax Tree (AST)** and runs it inside a custom-built TypeScript execution engine. This engine records a snapshot of all variables in scope, active stack frames, and heap addresses at every line of execution. This sequence of snapshots (called **traces**) is sent to the frontend, which serves as a state-driven playback system.

Real-World Analogy

* **LeetCode/Normal IDEs:** Like sending a letter to a factory and getting back a finished car. You don't know how it was built; you only see the final product.

* **CodeFlow:** Like watching a slow-motion video showing every single assembly line robot tightening bolts in order. If a robot makes a mistake, you can pause, rewind, and look at the exact line of code that commanded it.

Example Flow

1. **Problem selection:** A user selects the "Binary Search" problem from the curated DSA list.
2. **Visualization request:** The starter code loads in the editor. The user clicks **Visualize**.
3. **Simulation & Analysis:** The code goes to the backend where it is simulated step-by-step. The backend also queries AI in parallel to get an explanation of the logic.
4. **Interactive Playback:** The backend returns 25 execution steps. The frontend highlights line 6, draws two pointers (``low`` and ``high``), and displays: `"Checking the midpoint element at index 4."`

Interview Questions & Answers

* **Q: Why are existing coding platforms insufficient for beginners?**

* *A:* Existing platforms operate on a input-in/output-out model. They tell you *what* the result is, but not *how* it was reached. Debugging requires manually inserting print statements or setting up complex local debuggers. CodeFlow automates this by treating execution state as visual configurations.

* **Q: How does CodeFlow differ from tools like PythonTutor?**

* *A:* While PythonTutor creates linear memory tables, CodeFlow detects semantic intent (e.g., recognizing that an array is representing a binary heap) and renders custom, interactive widgets like trees, graphs, maps, and sliding windows. It also incorporates validation, auto-fixing, and AI-driven explanations.

2. COMPLETE TECH STACK

Simple Explanation

To build a house, you need structure (brick walls), styling (paint and decorations), logic (plumbing and electrical wiring), and communication (intercoms). CodeFlow uses React and Monaco for the structure and code input, Tailwind CSS and Framer Motion for styling and fluid animations, Node.js/WebSockets for logic and live communication, and MongoDB/Firebase to store user accounts.

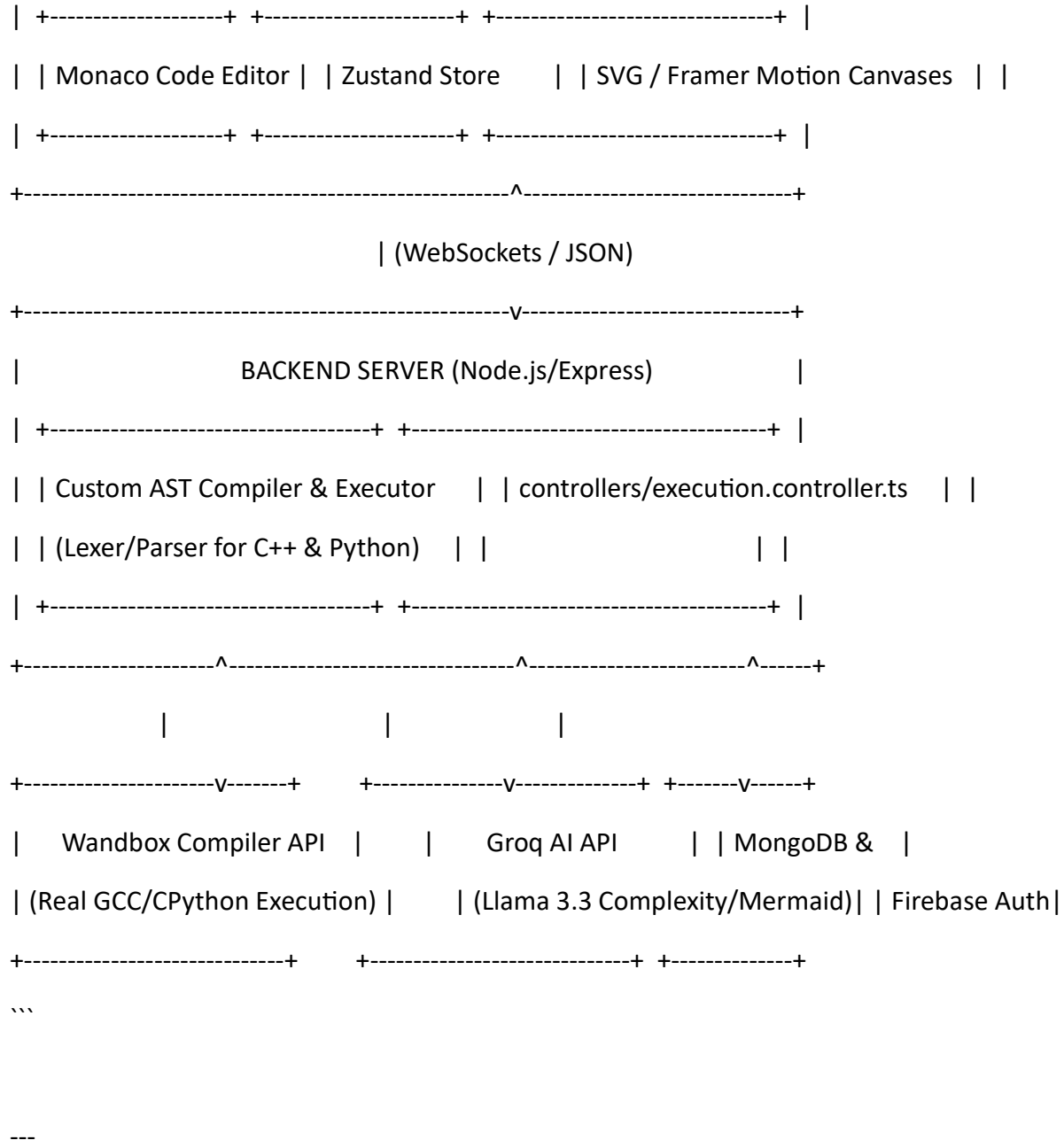
Technical Explanation

The complete tech stack is split between the frontend, backend, and external compiler APIs:

...

+-----+

| FRONTEND CLIENT (Vite/React) |



Frontend Components

React.js

* **Why we use it:** To build a modular, component-based user interface.

- * **Role:** Handles application routing (About, Dashboard, CuratedSheet, ProblemWorkspace) and synchronizes DOM updates based on application state.

TypeScript

- * **Why we use it:** C++ and Python syntax trees contain complex, nested structures. TypeScript catches type mismatch errors during compile time instead of failing in the browser.

- * **Role:** Dictates interfaces for the compiler AST, WebSocket responses, and visual representations (e.g., `[index.ts](file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/frontend/src/types/index.ts)`).

Tailwind CSS & Custom CSS

- * **Why we use it:** To build a premium dark-themed layout with high-performance glassmorphism styles without writing thousands of lines of boilerplate CSS.

- * **Role:** Handles general grid alignment, flexboxes, and responsive layouts. Custom CSS (`[visualizers.css](file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/frontend/src/features/visualizer/components/visualizers/visualizers.css)`) handles specialized element transformations like element swapping.

Monaco Editor

- * **Why we use it:** It is the exact editor engine that powers VS Code.

- * **Role:** Provides code highlighting, tab completions, auto-indentation, line numbers, and keyboard shortcuts directly in the browser.

Framer Motion

- * **Why we use it:** Standard CSS transitions look jumpy. Framer Motion uses spring physics to make moving pointers and shifting nodes feel fluid and natural.

- * **Role:** Animates pointer badges (e.g. ``low``, ``high``, ``mid``) and tree links smoothly as they slide between memory locations.

GSAP & Three.js

- * **Why we use it:** For high-performance graphics and entry animations that wow the user.
- * **Role:** Used to handle decorative landing pages and structural background canvas elements.

Backend Components

Node.js & Express.js

- * **Why we use it:** High concurrency, easy WebSocket handling, and a unified language (TypeScript) across the entire stack.
- * **Role:** Express handles user accounts, progress statistics, and GitHub OAuth redirects. Node.js processes the CPU-intensive code parsing and execution loop.

REST APIs & WebSockets

- * **Why we use it:** Standard HTTP requests are slow and require creating a new connection each time. WebSockets keep a persistent open pipe, allowing line-by-line trace steps to stream back to the UI.
- * **Role:** REST handles dashboard data; WebSockets handle the run, visualize, and auto-fix interactions.

Database Components

MongoDB (Mongoose)

- * **Why we use it:** Scalability and simple schema definition.

- * **Role:** Stores user records (emails, solved problem IDs, connected GitHub tokens, streaks).

Real-World Analogy

Imagine building an automated theatrical play:

- * **Monaco Editor:** The scriptwriter's typewriter.
- * **React:** The stage manager who sets up the curtains and rooms.
- * **Tailwind/Framer Motion:** The costume designer and puppet strings making movements look elegant.
- * **Zustand:** The script copyholder who makes sure everyone knows the current line.
- * **Node.js Backend:** The director shouting commands behind the scenes.
- * **WebSockets:** The walkie-talkie connecting the stage manager to the director.

Interview Questions & Answers

- * **Q:** Why use Zustand instead of Redux or React Context for this application?
 - * **A:** Redux has heavy boilerplate, and React Context causes full component tree re-renders, which would choke the animation loop. Zustand provides atomic state subscription, meaning only the visual canvas re-renders when the current trace step changes.
- * **Q:** Why did you implement WebSockets instead of a simple HTTP POST request to compile code?
 - * **A:** WebSockets allow us to stream responses. For long executions, we can send frames incrementally. It also allows us to build interactive protocols in the future (like prompting the user mid-execution for `cin` input).

3. HOW CODEFLOW WORKS (END-TO-END FLOW)

Simple Explanation

When a user selects a problem on the website, they see a split-screen layout with code on the left and a drawing canvas on the right.

Here is exactly what happens when you type a C++ program and click "Visualize":

1. ****Code Input:**** You type code in the editor.
2. ****WebSocket Send:**** The editor packages the code string and sends it through a fast websocket pipe to the backend.
3. ****Backend Check:**** The backend reads your code and checks for syntax bugs. If it finds one (like a missing `;`), it suggests a fix.
4. ****Custom Compiler:**** If it is valid, the backend's custom engine reads your code, translates it into instructions, and executes it line-by-line.
5. ****Snapshot Recording:**** While executing, the backend takes a photo (snapshot) of the variables and stack at every step.
6. ****AI Analysis:**** In parallel, the backend asks Groq AI to explain the time complexity and generate a flowchart.
7. ****Data Received:**** The backend packages the snapshots, flowchart, and complexity details into a single response and sends it to the frontend.
8. ****Interactive Playback:**** The frontend loads this data. When you click ****Play****, it loops through the snapshots, highlighting the active lines in the editor and updating the shapes on the canvas in real time.

Technical Explanation

The complete user interaction lifecycle consists of the following steps:

1. ****Frontend State Init:**** The user lands on
[ProblemWorkspace.tsx](file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/frontend/

src/pages/ProblemWorkspace.tsx). The state machine in [executionStore.ts](file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/frontend/src/store/executionStore.ts) is initialized.

2. **WebSocket Connection:** The client establishes a WebSocket connection to the backend server.

3. **Request Dispatch:** The user clicks "Visualize". The frontend sends a JSON payload:

```
```json
{
 "type": "TRACE",
 "payload": {
 "code": "#include <iostream> ...",
 "input": "5",
 "language": "cpp"
 }
}
```
```

4. **Backend Reception:** The WebSocket server in [server.ts](file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/backend/src/websocket/server.ts) forwards the message to [execution.controller.ts](file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/backend/src/controllers/execution.controller.ts#L85-L182).

5. **Validation Phase:** The backend calls [validation.service.ts](file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/backend/src/services/validation.service.ts). If syntax errors exist, they are compiled into `ValidationIssue` objects and returned to trigger the "Auto-Fix" panel.

6. **Trace Generation:** The compiler executor (e.g., `CppExecutor` in [executor.ts](file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/backend/src/engine/languages/cpp/executor.ts)) runs. It returns a Generator that yields a sequence of `ExecutionTrace` items.

7. ****AI Integration:**** The controller triggers parallel asynchronous tasks via ``AiService`` in `[ai.service.ts](file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/backend/src/service s/ai.service.ts)` to retrieve the code complexity structure and Mermaid flowchart.

8. ****Response Construction:**** The traces are combined with the AI analysis and sent back through the socket:

```
```json
{
 "type": "TRACE_RESULT",
 "payload": {
 "success": true,
 "steps": [...],
 "analysis": { ... }
 }
}
...

```

9. **\*\*Frontend Animation Loop:\*\*** The frontend receives the payload, populates ``traceSteps`` in the Zustand store, and triggers ``isPlaying = true``. A ``setInterval`` timer steps through each index, feeding data into the visualizers.

---

## ## 4. HOW THE CODE EDITOR WORKS

### ### Simple Explanation

Instead of building a text box from scratch, we embedded the **\*\*Monaco Editor\*\***—the same technology that powers Microsoft's VS Code. When you type C++ or Python code, it displays keywords (like ``for``, ``while``, ``int``) in distinct colors and offers autocomplete suggestions just like a local editor.

### ### Technical Explanation

- \* **Monaco Integration:** The frontend mounts the editor inside React. It loads styling sheets and sets up configurations like theme, font size, minimap options, and read-only flags.
- \* **Language Selection:** When the user toggles from C++ to Python, the state in the Zustand store changes, prompting Monaco to swap its language profile, updating syntax validation rules.
- \* **Active Line Highlighting:** The Monaco instance exposes a decoration API. The component tracks the `currentStepIndex`. When the step changes, the editor clears old decorations and highlights the line number matching `traces[currentStepIndex].line` with a custom CSS class.

### ### Real-World Analogy

It is like a digital music sheet. As the orchestra plays, a glowing laser pointer highlights the exact note that needs to be played next.

### ### Example Flow

1. **State Change:** The user clicks "Next Step" (step updates from index `4` to `5`).
2. **State Lookup:** The store reads `traces[5]`, which says the active line is `12`.
3. **Monaco Decoration Update:**

```
``typescript
editor.deltaDecorations(oldDecorations, [
 {
 range: new monaco.Range(12, 1, 12, 1),
 options: { isWholeLine: true, className: 'active-line-highlight' }
 }
]);
...

```

4. **UI Render:** Monaco renders a cyan-tinted highlight over line `12`.

---

## ## 5. HOW THE COMPILER WORKS

### ### Simple Explanation

A computer cannot run your code directly. It needs to read the text, translate it into structure, and run it. CodeFlow has two ways of running your code:

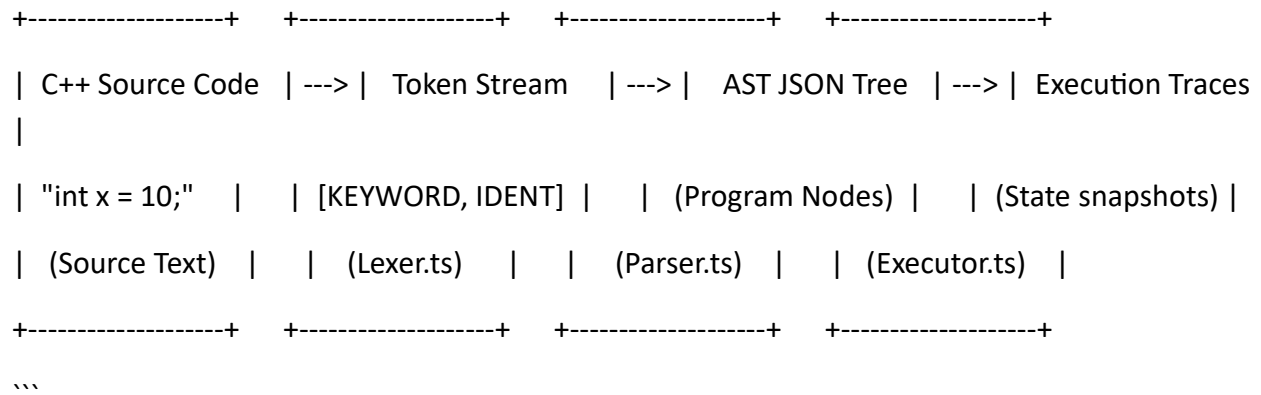
1. **The Custom Simulator (Interpreter):** We wrote a custom-made program in TypeScript that acts like a micro-compiler. It reads C++ or Python code, parses it into an instruction tree, and executes it line-by-line to extract intermediate variables for visualization.
2. **The Real Compiler (Wandbox):** For real code output, it uses the Wandbox API, which executes the code on a real remote server inside a secure, sandbox environment.

### ### Technical Explanation

The custom engine in ``backend/src/engine`` is a **Tree-Walking Interpreter**. Let's break down how the C++ parser works in `[parser.ts](file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/backend/src/engine/languages/cpp/parser.ts)`:

- \* **Lexical Analysis (Lexer):** Converts the source string into a stream of tokens. It uses regular expressions to match C++ keywords, identifiers, literals, operators, and punctuation.
- \* **Parser:** Iterates through the tokens using recursive descent parsing. It parses structures into standard AST Node objects representing variable declarations, assignments, class definitions, function calls, conditional branches, and loops.
- \* **AST Execution:** The ``Executor`` class in `[executor.ts](file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/backend/src/engine/languages/cpp/executor.ts)` contains visitor functions. When it visits a ``ForStatement``, it executes the initializer, evaluates the loop condition, executes the loop body, runs the update expression, and loops until the condition fails.

...



### ### Real-World Analogy

Imagine receiving a recipe written in a foreign language (C++ code):

1. **\*\*Lexer:\*\*** Translates the letters into recognized food items and quantities (eggs, flour, milk).
2. **\*\*Parser:\*\*** Organizes the recipe into logical steps (First mix ingredients, then bake, then cool).
3. **\*\*Executor:\*\*** You (the chef) actually cook the recipe step-by-step, taking a photo of the mixing bowl at every step.

---

## ## 6. HOW RUN CODE FEATURE WORKS

### ### Simple Explanation

When you want to check if your code runs correctly against standard inputs (without step-by-step animations), you click **\*\*Run Code\*\***. The backend acts as an intermediary, sending your code to a secure compilation server (Wandbox) and returning the printed outputs.

### ### Technical Explanation

\* **Endpoint Communication:** The client dispatches a WebSocket message with type ``RUN_CODE``.

\* **Wandbox Invocation:** The controller invokes ``CompilerService.execute()`` in `[compiler.service.ts](file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/backend/src/services/compiler.service.ts)`.

\* **Payload Structuring:** The compiler service maps the requested language to Wandbox compiler names (e.g. ``cpp`` maps to ``gcc-head``, ``python`` to ``cpython-head``) and posts a request:

```
``typescript
```

```
const payload = {
```

```
 compiler: 'gcc-head',
```

```
 code: sourceCode,
```

```
 stdin: inputData,
```

```
 options: 'warning,gnu++17'
```

```
};
```

```
const response = await axios.post('https://wandbox.org/api/compile.json', payload);
```

```
...
```

\* **Result Mapping:** The returned response contains keys like ``program_message``, ``program_error``, and ``status``. The server normalizes this into a ``RunResult`` object and returns it to the client.

### ### Real-World Analogy

It is like using Google Translate. You write text, send it to Google's translation server, it processes it, and returns the result back to your screen.

---

## ## 7. HOW EXECUTION TRACE WORKS

### ### Simple Explanation

To animate a movie, you need individual film cells. An **execution trace** is a collection of these film cells. Every time the code jumps to a new line, the executor records a snapshot of the current state of variables in memory. When these snapshots are played back in order, you see your code execute step-by-step.

### ### Technical Explanation

The trace generator in

[`executor.ts`](`file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/backend/src/engine/languages/cpp/executor.ts`) records snapshots of the stack, heap, and output buffer at every line change.

\* **Stack Representation:** Variables are stored in an array of `Environment`` objects. Each environment represents a stack frame containing local variables.

\* **Heap Representation:** Objects (like tree nodes or custom structures) are allocated memory address references (e.g. `#1000``, `#1001``). The executor maps references to key-value objects in a global `heap`` record.

\* **Trace Snapshot Structure:**

```
``typescript
```

```
interface ExecutionTrace {
```

```
 line: number;
```

```
 type: 'assignment' | 'condition' | 'function_call' | 'return';
```

```
 stack: StackFrame[];
```

```
 heap: Record<string, any>;
```

```
 output: string;
```

```
 visualization?: VisualizationHint;
```

```
 visuals?: any;
```

```
}
```

...

### ### Example Flow

Let's trace:

```cpp

1: int a = 5;

2: a = a + 2;

...

* **Line 1 Execution:**

- * Executor defines `a` as `5` in the current environment.

- * Yields trace step: `line: 1, type: "definition", stack: [{ function: "main", locals: { a: 5 } }]`.

* **Line 2 Execution:**

- * Executor evaluates `a + 2` -> `7`. Assigns `a` to `7`.

- * Yields trace step: `line: 2, type: "assignment", stack: [{ function: "main", locals: { a: 7 } }]`.

8. HOW VISUALIZATION IS CREATED

Simple Explanation

How does the computer know that a variable is an array, list, or tree? It reads the variable type. If the compiler executes `int arr[10];`, it flags that variable as an **Array**. If it executes a custom class with `left` and `right` pointer variables, it recognizes it as a **Tree**. The visualization engine takes these data structures and uses React widgets to draw them.

Technical Explanation

The execution engine uses type reflection and variable tracking in the visitor classes of [executor.ts](file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/backend/src/engine/languages/cpp/executor.ts):

1. **Type Extraction:** During declaration (`VariableDeclaration``), the parser extracts the C++ type name string (e.g. `vector<int>`, `stack<int>`, `TreeNode*`).
2. **Detection Mapping:** The executor maps the type string to visual types:
 - * `isVector()`, `isStack()`, `isQueue()`, `isPriorityQueue()` evaluate the type strings to map them to visual layout modes.
3. **Semantic Visual Object Generation:** During execution, variable updates compile custom data structures. For example:
 - * A C++ `priority_queue` is compiled into a JSON representation containing the elements array and a heap-type flag.
4. **Frontend State Subscription:** The frontend reads the `visuals` field in the current trace step and mounts the matching visualizer component.

9. HOW SHAPES ARE DRAWN

Simple Explanation

To draw data structures on the screen, we map memory variables directly to HTML/SVG templates.

- * **Arrays:** Drawn as boxes lined up in a row.
- * **Stacks/Queues:** Drawn as horizontal or vertical boxes where elements slide in or out.
- * **Linked Lists:** Drawn as boxes with arrows connecting them.
- * **Trees:** Drawn as parent circles with child circles below them.

Technical Explanation

The visual canvas uses specialized React components inside
`frontend/src/features/visualizer/components/visualizers/`:

* **Array Visualizer

([ArrayVisualizer.tsx](file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/frontend/src/features/visualizer/components/visualizers/ArrayVisualizer.tsx)):** Loops through values and draws flexbox cells. It supports standard grid layouts (Blocks mode) and variable heights matching the numerical value (Bars mode for sorting algorithms).

* **Tree Visualizer

([TreeVisualizer.tsx](file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/frontend/src/features/visualizer/components/visualizers/TreeVisualizer.tsx)):** Traverses nodes and uses a memoized layout manager to compute XX and YY coordinates. Parent nodes are placed centered above siblings. SVG `` elements draw edges between coordinates.

* **Memory Graph Visualizer

([MemoryGraphVisualizer.tsx](file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/frontend/src/features/visualizer/components/visualizers/MemoryGraphVisualizer.tsx)):** Automatically maps pointer addresses. It reads the position of stack variables and heap blocks using `getBoundingClientRect()` and draws animated SVG curves connecting them.

...

```
+-----+
| SVG CANVAS                                     |
|                                               |
|                                               |
| (Root Node Position: x:300, y:60)           |
| [Node: 10]                                   |
| / \ <--- (SVG <line> drawn from (300,60) to (200,130)) |
| / \                                         |
| [Node: 5] [Node: 15]                         |
| (x:200, y:130) (x:400, y:130)               |
```

+-----+
...

Real-World Analogy

It is like using a coordinates blueprint to build a playground. The layout algorithm calculates where to put the slides and swings, and construction workers connect them with paths (lines) so children can play.

10. HOW LIVE VISUALIZATION WORKS

Simple Explanation

When you click ****Play****, the code visualization starts moving. The visualizer page syncs the highlighted code line with the drawing canvas. As the highlighted line moves down the page, the shapes on the right update their positions and values instantly, creating a live execution stream.

Technical Explanation

The synchronization is managed by a React state hook subscribing to the Zustand store in [executionStore.ts](file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/frontend/src/store/executionStore.ts):

1. ****State Initialization:**** Clicking "Visualize" triggers ``runCode()``, which connects to the backend and populates ``traces`` inside the store.
2. ****Interval Trigger:**** ``togglePlay()`` initializes a JavaScript interval loop:

```
``typescript
```

```
intervalId = setInterval(() => {  
  get().nextStep();
```

```
}, speed);
```

```
...
```

3. **Atomic Updates:** `nextStep()` increments `currentStepIndex`.

4. **Reactive Re-rendering:** Components like `ArrayVisualizer` subscribe to `traces[currentStepIndex]`. When the index changes, React updates the canvas nodes, triggering Framer Motion transition animations.

```
---
```

11. HOW GROQ AI IS USED

Simple Explanation

While our simulator runs code step-by-step, it doesn't know *why* you wrote the code. It doesn't understand your intentions. We use **Groq AI** (powered by Llama 3.3) to read your code and act as a teacher. It explains the algorithm's pattern, analyzes time complexity, and draws flowchart diagrams of the logic.

Technical Explanation

AI interactions are handled in

[ai.service.ts](file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/backend/src/services/ai.service.ts):

* **API Configuration:** The backend initializes the Groq client:

```
``typescript
```

```
this.groq = new Groq({ apiKey: this.apiKey });
```

```
...
```

* **Analysis Prompt:** The server sends a structured system prompt requesting a detailed JSON payload containing complexity analysis, learning comparisons, and step-by-step derivations.

- * **Mermaid Flowchart Generation:** Groq reads the code structure and outputs raw Mermaid flowchart diagrams (``graph TD``).

- * **AI vs. Deterministic Splitting:**

- * **Deterministic:** Variable values, memory tables, call stack frames, and console outputs are generated locally on the backend.

- * **AI-Generated:** Narrative explanations, flowchart diagrams, and Big-O explanations.

Real-World Analogy

Imagine studying with two friends:

- * **Friend A (The Interpreter):** Excellent at math. They compute equations exactly but cannot explain why the math is useful.

- * **Friend B (Groq AI):** A teacher who explains the concepts, draws diagrams on the board, and compares your solution to faster alternatives.

12. HOW TIME COMPLEXITY IS CALCULATED

Simple Explanation

To calculate how fast your code runs (Time Complexity) and how much memory it uses (Space Complexity), CodeFlow analyzes the structure of the code. It checks for loops, nested loops, and recursive calls. If there are no loops, it is constant time $O(1)$. A single loop is linear time $O(N)$, and nested loops are quadratic time $O(N^2)$. We also ask Groq AI to double-check the calculations for complex code.

Technical Explanation

CodeFlow uses a hybrid approach: **Deterministic AST Heuristic Parser** and **LLM verification**. Let's review the heuristic engine in

[ai.service.ts](file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/backend/src/services/ai.service.ts#L185-L721):

- * **Syntax Check:** Regular expressions inspect references to collections (vectors, lists, maps).
- * **Loop Nesting Calculation:** The engine counts nested loop blocks by inspecting structural parser brackets:
 - * No loops -> $O(1)$
 - * Single loop -> $O(N)$
 - * Two nested loops -> $O(N^2)$
 - * Three nested loops -> $O(N^3)$
- * **Binary Search Detection:** Looks for bounds-halving logic (e.g. `mid = (low + high) / 2`), matching it to $O(\log N)$ complexity.
- * **Recursion Detection:** Scans function declarations for recursive calls. If it detects subproblem division, it maps it to $O(N \log N)$ or $O(N)$.

13. HOW RECURSION TREE IS GENERATED

Simple Explanation

When a function calls itself, it creates a chain of calls. If you calculate the Fibonacci sequence of `5` recursively, it calls Fibonacci of `4` and `3`. Each of those calls others, forming a tree-like shape. We track each function call, recording who called whom, and draw them on a canvas so you can watch the recursion tree grow and watch return values propagate.

Technical Explanation

- * **Frame Association:** When the AST Executor visits a `CallExpression` inside a function with the same name, it increments `callDepth`.

* ****Parent-Child Mapping:**** The executor generates a unique identifier for the current invocation node. It records the active parent node ID from the call stack frame.

* ****Tree Visualization Node Output:**** During execution, a node is appended to the tree visuals list:

```
``typescript
nodes.push({
  id: currentNodeId,
  value: argumentValue,
  parentId: parentNodeId
});
...

```

* ****Trace Render:**** The tree layout calculates position coordinates, and edges connect caller functions to their sub-calls, animating elements as return values are returned back to parent frames.

Real-World Analogy

It is like a family tree. Every time a new child function is born, we draw a line from the parent. When the child finishes their task (returns a value), they report it back to the parent.

14. HOW DP VISUALIZATION WORKS

Simple Explanation

Dynamic Programming (DP) is like writing down intermediate results on a notepad so you don't have to re-calculate them. CodeFlow visualizes this by showing a grid representing your cache (notepad). When a subproblem is calculated, the cell lights up with the result. If the code tries to calculate it again, it retrieves it from the grid (Cache Hit), preventing redundant calculation branches.

Technical Explanation

- * **Cache Extraction:** The execution compiler tracks array declarations or mapping tables designated as DP tables (e.g., arrays named ``memo`` or ``dp``).
- * **Read/Write Hooking:** Every time an AST node executes a ``MemberExpression`` write (e.g. ``dp[i] = val``) or read (e.g. ``return dp[i]``), the executor flags the operation.
- * **State Animation:**
 - * **Cache Write:** The grid cell flashes orange, displaying the saved value.
 - * **Cache Read (Hit):** The grid cell flashes green, indicating that the value was retrieved instantly from memory.

Example Flow (Fibonacci DP)

1. **Request:** User runs recursive Fibonacci with memoization for ``N = 4``.
2. **Simulation:**
 - * ``fib(2)`` is computed. Backend saves it in cache: ``dp[2] = 1``.
 - * Later, ``fib(3)`` calls ``fib(2)``. The executor executes the check: ``if (dp[2] != -1)``.
 - * ``dp[2]`` is accessed. The executor records a trace step indicating a **Cache Hit** at index ``2``.
3. **Render:** The frontend DP grid flashes green at index ``2``.

15. HOW GITHUB INTEGRATION WORKS

Simple Explanation

You can connect your GitHub account to CodeFlow. This lets you import your own code files directly from your GitHub repositories into the Monaco Code Editor. It uses GitHub OAuth to login securely without sharing your password.

Technical Explanation

The repository management is handled in

[github.controller.ts](file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/backend/src/controllers/github.controller.ts):

* **OAuth Authorization:** The user clicks "Connect GitHub". The backend redirects to:

`https://github.com/login/oauth/authorize?client\_id=CLIENT\_ID&redirect\_uri=CALLBACK\_URL&scope=repo,user`

* **Code Exchange:** After approval, GitHub redirects back to

`/api/github/callback?code=CODE`. The server posts to GitHub's token exchange endpoint:

```
``typescript
```

```
const tokenResponse = await axios.post('https://github.com/login/oauth/access_token', {  
  client_id: clientId,  
  client_secret: clientSecret,  
  code  
});  
...
```

* **Persistence:** The resulting `access\_token` is saved to the user's MongoDB record.

* **API Proxying:** When the user imports a file, the backend fetches the repository list or file contents from `api.github.com/repos/{repo}/contents/{path}` using the user's saved access token, and returns the contents back to the frontend.

16. HOW USER PROGRESS IS STORED

Simple Explanation

When you complete problems, CodeFlow saves your achievements, Streak points, and solved progress. All this information is tied to your account and stored in a secure database. Even if you log out or switch computers, your progress is loaded automatically when you sign back in.

Technical Explanation

* **User Schema:** The database model

[User.ts](file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/backend/src/models/User.ts) stores registration dates, completed problem lists, code templates, streaks, and activity history.

* **Progress Incrementation:** When a user completes a problem successfully, the client posts to `/api/users/progress`. The backend appends the problem ID to the solved list and updates active streak logs.

* **Dashboard Restoration:** On login, the user profile is returned in the API payload, restoring visual badges and streaks on the frontend dashboard.

17. HOW THE BACKEND WORKS (ARCHITECTURE)

Simple Explanation

The backend acts like the nervous system of the application. It receives requests from the frontend, routes them to the right controller (e.g., verifying user accounts, fetching problems, running code), queries the database, and sends the results back to your screen.

Technical Explanation

The backend uses a standard **Model-View-Controller (MVC)** architectural pattern written in TypeScript:

...



18. HOW THE FRONTEND WORKS (ARCHITECTURE)

Simple Explanation

The frontend handles everything the user sees. It routes you to different pages, displays the code editor and animation canvas, and uses a centralized store (Zustand) to manage execution states so that buttons, editor lines, and visual shapes stay in sync.

Technical Explanation

The frontend codebase layout is organized by feature modules:

- * **Pages (`src/pages/`):** Contains entry pages like [Dashboard.tsx](file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/frontend/src/pages/Dashboard.tsx) and [ProblemWorkspace.tsx](file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/frontend/src/pages/ProblemWorkspace.tsx).
- * **State Store (`src/store/`):** Managed by Zustand.
 - * [executionStore.ts](file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/frontend/src/store/executionStore.ts) handles visual execution parameters, active indexes, playback state, and socket states.
 - * `authStore.ts` handles user accounts.
- * **Visualizer Features (`src/features/visualizer/`):** Contains visualizer canvas widgets like `ArrayVisualizer` and `TreeVisualizer` which convert trace values into responsive SVG shapes.

19. HOW BACKEND SENDS DATA TO FRONTEND

Simple Explanation

To communicate, the backend packs all the steps, variables, and drawings into a structured JSON string and sends it over the WebSocket. The frontend reads this string, parses it, and loops through it to animate the changes.

Technical Explanation

Let's review the exact JSON payloads:

Trace Step Payload (Sent from Backend to Frontend)

```
```json
{
 "type": "TRACE_RESULT",
 "payload": {
 "success": true,
 "totalSteps": 3,
 "steps": [
 {
 "step": 1,
 "line": 4,
 "lineContent": "int x = 5;",
 "variables": { "x": 5 },
 "visuals": {
 "type": "array_1d",
 "target": "arr",
 "values": [5, 2, 8]
 }
 },

```

```

 "teacherNote": {
 "what": "Initialized variable x to 5",
 "why": "First step of array scan",
 "next": "Compare element at index 0"
 }
}
],
"analysis": {
 "timeComplexity": "O(N)",
 "spaceComplexity": "O(1)",
 "pattern": "Linear Scan"
}
}
}
...

```

## ## 20. HOW TESTING WORKS

### ### Simple Explanation

If we make a change to our compiler, how do we know we didn't break C++ or Python support? We wrote a test runner. It loads a database of **\*\*200 real DSA problems\*\***, runs our simulator against every single one of them, and checks that they execute correctly, generate the right steps, and draw the correct data structures.

### ### Technical Explanation

The test runner is configured in

[test\_sheet\_problems.ts](file:///c:/Users/asati/OneDrive/Desktop/CODE%20Visualizer/backend/src/test\_sheet\_problems.ts):

\* **Curated Data Scraping:** The script searches `problems` folders and extracts `starterCode`.

\* **Mock Execution Loop:** For each file, the runner spins up an execution instance:

```
``typescript

const executor = new Executor();

const generator = executor.execute(sourceCode, "");

...

```

\* **Constraint Checking:** The runner asserts that:

1. The generator executes without raising exceptions.
2. The steps generated do not exceed `1500` (which would indicate an infinite loop).
3. The output contains visualizer objects.

\* **Validation Assertion:** If all 200 items complete execution, the suite exits successfully.

---

## ## 21. DEPLOYMENT

### ### Simple Explanation

To make the site public on the internet, we host the frontend on **Vercel** (for fast page load speeds) and the backend server on a hosting service like **Render** or **AWS**. The MongoDB database runs in the cloud on **MongoDB Atlas**. They communicate securely over HTTP and WebSockets.

### ### Technical Explanation

- \* **Frontend Hosting (Vercel):** The frontend build task executes ``npm run build`` which compiles TypeScript into static JS bundles. Vercel serves these files over a global CDN.
- \* **Backend Hosting (Render/AWS ECS):** The Node.js server is packaged and deployed. It exposes port ``5000`` to handle Express API endpoints and WebSocket handshakes.
- \* **Environment Syncing:** Client requests route via environment variables:
  - \* Frontend ``VITE_API_URL`` points to the hosted server.
  - \* Backend ``GROQ_API_KEY`` and ``MONGODB_URI`` securely connect services in production.

---

## ## 22. COMMON INTERVIEW QUESTIONS

### ### Why is CodeFlow technically challenging?

- \* **Answer:** CodeFlow is technically challenging because it doesn't just run code—it interprets it in-memory. Building a custom lexer, parser, and runtime environment in TypeScript that mimics C++ STL structures (like stacks, queues, and heaps) and records execution frame states at every step is a complex task. Additionally, mapping memory addresses to coordinate-based UI nodes (like recursion trees and pointer arrows) requires high-performance layout algorithms and synchronization.

### ### What was the hardest part?

- \* **Answer:** The hardest part was building the **Memory Graph Visualizer** and drawing the reference arrows between the stack and the heap. Because stack and heap frames are standard HTML elements that render dynamically in a scrollable list, calculating their absolute pixel offsets on screen resize or trace step updates and rendering interactive SVG Bezier curves without lagging the render engine required complex coordinate math and React rendering optimizations.

### ### What AI components were used?



\* **Answer:** We used the **Groq API** with the **Llama 3.3 (70B)** model. It acts as a parallel assistant:

1. It performs static analysis to determine code pattern categories.
2. It calculates asymptotic time and space complexity ( $O(N)$  vs  $O(N^2)$ ).
3. It generates flowcharts in Mermaid.js syntax dynamically.

All data execution traces (variable updates, step-by-step numbers) are kept **deterministic** (calculated locally) to ensure correctness.

### What scalability challenges exist?

\* **Answer:** The primary scalability challenge is CPU execution overhead on the backend. Because we parse and interpret code inside a single-threaded Node.js engine, running large loops (e.g., millions of iterations) would block the event loop. We mitigate this by restricting visual executions to a maximum of 2,000 steps and using AST limits. If we scale up, we would offload interpreter tasks to worker threads or compute containers.

### What would you improve next?

\* **Answer:**

1. **Multi-File Execution:** Support importing and linking multiple header and source files.
2. **State-Saving Sessions:** Allow users to save their execution trace walkthroughs and share links with notes for classroom settings.
3. **Expanded Language Support:** Extend the AST engine to support Java and Javascript syntaxes.

### What would happen if 10,000 users use it simultaneously? How would you scale it?

\* **Answer:** If 10,000 users call the backend simultaneously, the single-threaded Node.js process would run out of CPU and memory, causing WebSockets to disconnect.

To scale it:

1. **Horizontal Scaling:** Run multiple instances of the backend server behind a load balancer (e.g., AWS ALBs or Nginx) using round-robin routing.

2. **Redis Adapter:** Integrate a Redis Pub/Sub adapter to manage WebSocket session states across multiple server instances.

3. **Serverless Execution Workers:** Offload the AST interpretation step to serverless functions (like AWS Lambda) to keep the main Express server lightweight and responsive.